



Prática II: Gerência de Memória

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

Informações

- Crie um documento de texto para responder as perguntas e inserir as capturas de tela requeridas durante o experimento
- Pesquise sobre as questões envolvendo os conceitos de memória

1 Objetivos

Compreender a divisão da memória em blocos de tamanho fixo (páginas), observar o mapeamento de endereços virtuais para físicos no Linux, analisar o comportamento do Kernel frente à paginação, examinar estatísticas globais de páginas do sistema e compreender o funcionamento do mecanismo de *Swap*.

2 Roteiro Prático II - Hardware de Memória

1. Descobrindo o Tamanho da Página e Limites do Sistema

Antes de começarmos a alocar memória, precisamos saber qual é o tamanho de página (*page size*) configurado no hardware e pelo kernel do sistema operacional, além de investigar limites lógicos do subsistema de memória.

Abra o terminal do Linux e execute os comandos abaixo:

```
getconf PAGESIZE  
sysctl kernel.shmall
```

Nota: O parâmetro `shmall` define a quantidade total de páginas de memória compartilhada que o sistema pode alocar simultaneamente.

Perguntas:

- Qual foi o valor retornado em bytes para o tamanho de página? Converta esse valor para KBytes (KB). O que esse número representa na arquitetura de paginação?
- Multiplique o valor obtido em `kernel.shmall` pelo tamanho da página em bytes. Qual é o limite máximo de alocação de memória compartilhada em Gigabytes (GB) configurado no seu sistema?

2. Alocação Dinâmica e Páginas Virtuais vs. Físicas

O Linux utiliza uma técnica chamada *Lazy Allocation* (Alocação Preguiçosa). Quando um programa solicita memória usando `malloc`, o sistema reserva as páginas no espaço virtual, mas adia a associação de quadros (*frames*) físicos na RAM até que o programa realmente tente escrever dados nelas, gerando uma falta de página.

- Crie um arquivo chamado `paginacao.c` e adicione o código abaixo:

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
  
#define DEZESSEIS_MEGA (16 * 1024 * 1024) // 16 MB
```

```

int main() {
    int pid = getpid();
    printf("=== PROCESSO INICIADO (PID: %d) ===\n", pid);
    printf("Passo A: Antes de alocar memoria. Verifique o consumo.\n");
    printf("Pressione ENTER para alocar 16 MB (mas sem usar)...");
    getchar();

    // Alocando 16 Megabytes de memoria virtual
    char *memoria = (char *)malloc(DEZESSEIS_MEGA);
    if (memoria == NULL) {
        perror("Erro ao alocar");
        return 1;
    }

    printf("\nPasso B: 16 MB alocados virtualmente.\n");
    printf("Pressione ENTER para ESCREVER nas paginas de memoria...");
    getchar();

    // Escrevendo em cada pagina (pulando de 4096 em 4096 bytes)
    // Isso força o Kernel a alocar os quadros fisicos na RAM
    for (int i = 0; i < DEZESSEIS_MEGA; i += 4096) {
        memoria[i] = 'X';
    }

    printf("\nPasso C: Dados escritos na memoria (Paginas fisicas associadas).\n");
    printf("Pressione ENTER para encerrar o programa...");
    getchar();

    free(memoria);
    return 0;
}

```

3. Compilação e Monitoramento com ferramentas ps e /proc

- Compile o programa executando o comando:

```
gcc paginacao.c -o paginacao
```

- Para monitorar o impacto desse programa na memória, abra **dois terminais** lado a lado no seu ambiente Linux. No **Terminal 1**, execute o programa:

```
./paginacao
```

- Anote o número do PID impresso na tela. No **Terminal 2**, use os utilitários do sistema para coletar o tamanho da memória virtual (VSZ / VmSize) e o tamanho físico residente na RAM (RSS / VmRSS) em KBytes.
- Execute os comandos abaixo no **Terminal 2** (substitua [PID] pelo número real):

```
ps -o pid,vsz,rss,comm -p [PID]
cat /proc/[PID]/status | grep -E "VmSize|VmRSS"
```

- Avançando o programa no Terminal 1 pressionando ENTER, execute os comandos do Terminal 2 para cada etapa e preencha a tabela a seguir:

Momento do Programa	VSZ / VmSize (KB)	RSS / VmRSS (KB)
Passo A (Início)		
Passo B (malloc feito)		
Passo C (Escrita feita)		

- **Perguntas:**

- Quando o `malloc` foi executado (Passo B), o que aconteceu com o valor da Memória Virtual? E com a Memória Física? Explique o motivo.
- Após o laço `for` escrever nas posições (Passo C), o que aconteceu com o espaço físico (RSS)? Explique por que pulamos exatamente de 4096 em 4096 bytes no código fonte.

4. Inspeccionando a Tabela de Páginas com o `pmap`

Mantenha o programa em execução parado exatamente no **Passo C**. No **Terminal 2**, execute o utilitário de mapeamento de processos:

```
sudo apt-get update
sudo apt-get install build-essential pmap

pmap -x [PID]
```

Pergunta: Procure na saída do comando a linha correspondente à alocação efetuada (procure por algo próximo a 16384K). Anote os valores das colunas Kbytes, RSS e Dirty. Pesquise o que a coluna Dirty (Suja) indica no contexto de gerência de páginas pelo Sistema Operacional.

5. Análise de Páginas Globais e Atividade de Swap (`/proc/meminfo` e `vmstat`)

A paginação gerencia tanto a memória dos processos quanto as páginas de cache de arquivos do sistema. Vamos extrair métricas globais em tempo real.

- Execute o comando no terminal para inspecionar os contadores de páginas de cache de disco mantidos pelo Kernel:

```
cat /proc/meminfo | grep -E "Cached|Buffers"
```

- Agora, ative o utilitário `vmstat` para atualizar as estatísticas de desempenho a cada 2 segundos:

```
vmstat 2
```

- Mantendo o terminal com o `vmstat` visível, abra uma aplicação pesada (como um navegador de internet) ou manipule um arquivo grande. Observe as alterações nas colunas sob os blocos `memory` (`buff`, `cache`) e `io` (`bi`, `bo`).

- **Perguntas:**

- O que indicam especificamente as colunas `si` (*swap in*) e `so` (*swap out*) localizadas sob o cabeçalho `swap`?
- Se você executar apenas `cat /proc/meminfo` encontrará linhas relacionados a um termo chamado **HugePages**. Pesquise o que esse termo significa.

6. Mapeamento de Arquivos em Páginas (`mmap`)

Uma das grandes utilidades do conceito de paginação é permitir mapear arquivos em disco diretamente no espaço de endereçamento virtual de um processo através da *system call* `mmap`.

- Crie primeiramente um arquivo de texto para testes usando o terminal:

```
echo "Arquitetura de Computadores e Sistemas Operacionais" > texto.txt
```

- Agora, crie um arquivo chamado `mapeado.c` com o código abaixo:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    int fd = open("texto.txt", O_RDWR);
    if (fd < 0) {
        perror("Erro ao abrir arquivo");
        return 1;
    }

    struct stat sb;
    fstat(fd, &sb);

    // Mapeando o arquivo diretamente em uma pagina de memoria virtual
    char *arquivo_em_memoria = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    if (arquivo_em_memoria == MAP_FAILED) {
        perror("Erro no mmap");
        return 1;
    }

    // O arquivo agora pode ser lido como se fosse um vetor na memoria
    printf("Conteudo lido da pagina de memoria: %s", arquivo_em_memoria);

    // Modificando a memoria diretamente. A MMU e o Kernel sincronizarao com o disco.
    arquivo_em_memoria[0] = '0';

    printf("Modificacao feita na memoria. Pressione ENTER para finalizar...");
    getchar();

    munmap(arquivo_em_memoria, sb.st_size);
    close(fd);
    return 0;
}

```

- Compile e execute o programa:

```

gcc mapeado.c -o mapeado
./mapeado

```

- Após o encerramento do programa, verifique o conteúdo final do seu arquivo de texto executando:

```

cat texto.txt

```

Perguntas: O que mudou no arquivo `texto.txt`? Pesquise como a paginação permitiu alterar um arquivo em disco sem que tivéssemos que chamar funções explícitas de escrita em arquivo (como `write` ou `fprintf`).

7. Podemos monitorar paginação em tempo real:

```

sar -B 1 10

```

O comando coleta estatísticas a cada 1 segundo, por 10 vezes. O mesmo faz parte do pacote **sysstat** que funciona como um “caixa-preta” do Linux: ele coleta estatísticas de CPU, memória, disco, rede e swap em tempo real e também grava históricos para análise posterior. **Caso você tenha acesso como super-usuário** basta instalar via **sudo apt install sysstat**. Pesquise o que cada linha da resposta significa.

3 Roteiro Prático II - Uso de Memória

1. Crie o arquivo hello.c:

```
#include <stdio.h>
#include <unistd.h>
int global_init = 42; // DATA
int global_uninit;    // BSS
int main() {
    printf("Hello World!\n");
    sleep(600);
}
```

2. Compile e rode:

```
gcc hello.c -o hello
./hello &
ps aux | grep hello # descubra o PID
pmap <PID>
```

3. Exemplo de alocação estática, automática e dinâmica

```
#include <stdio.h>
#include <stdlib.h>
int global = 10; // Estática
void funcao_auto(int n) {
    int local = n; // Automática
    printf("Local = %d\n", local);
}

int main() {
    funcao_auto(5);
    int *ptr = malloc(sizeof(int)); // Dinâmica
    *ptr = 99;
    printf("Dinamica = %d\n", *ptr);
    free(ptr);
    return 0;
}
```

4. Compile e rode:

```
gcc alocacao.c -o alocacao
./alocacao
```

5. Vamos analisar um exemplo recursivo: Crie **fatorial.c**:

```
#include <stdio.h>

long fatorial(int n) {
    long parcial;
    printf("inicio: n = %d\n", n);
    if (n < 2)
        parcial = 1;
    else
        parcial = n * fatorial(n - 1);
    printf("final : n = %d, parcial = %ld\n", n, parcial);
    return parcial;
}

int main() {
    printf("Fatorial(4) = %ld\n", fatorial(4));
    return 0;
}
```

6. Compile e rode:

```
gcc fatorial.c -o fatorial
./fatorial
```

Analise que cada chamada cria novas variáveis locais na STACK, como pratos empilhados. Ao retornar, os pratos são retirados.

4 Considerações Finais para o Relatório

Baseando-se nos seus experimentos e nos conceitos de hardware de memória vistos em aula, explique com suas palavras qual é a função da **MMU (Memory Management Unit)**.